Git & GitHub Cheatsheet

by Abhishek Kashyap
Placement-Ready Developer Notes

■ github.com/abhishekkashyap02 | linkedin.com/in/abhishek-kashyap-6b283322b

What is Git? What is GitHub?

■ Git

Git is a free, open-source **Version Control System** that runs locally. It tracks every change you make and lets you go back to any older version — like a **time machine for your code**.

■■ GitHub

GitHub is a **cloud platform** to host your Git repos online. Share code, collaborate with teams, raise Pull Requests, and show your work to recruiters — like **Google Drive but for code**.

■ Why it matters

Every company uses Git. Interviewers check your GitHub. Without Git, team collaboration is chaos. Git + GitHub = **must-have skill** for every developer.

Feature	Git	GitHub
Runs on	Your local machine	Cloud (online)
What it is	Version control tool	Hosting platform for repos
Needs internet	No	Yes
Made by	Linus Torvalds (2005)	Microsoft (acquired 2018)
Free?	Yes, always	Free for public repos
Used for	Track changes, branch, merge	Share, collaborate, deploy

Section 1 — Setup & Configuration

■■ **First Time Setup:** Install Git from **git-scm.com**. Then configure your identity — Git tags each commit with your name and email so teammates know who made which change.

■■ Setup & Configuration

`git --version`	**Check if Git is installed** and see the version number.
`git config --global user.name "Name"`	**Set your name globally.** Appears on all commits. Use --local for per-project.
`git config --global user.email "e@x"`	**Set your email.** Must match your GitHub email so commits link to your profile.
`git config --list`	**View all config settings** — name, email, editor, etc.
`git config --global core.editor "code --wait"`	**Set VS Code as default editor** for commit messages.

■ Run global config only once. It applies to all future repos on your machine.

Section 2 — Creating & Cloning a Repository

■ **Repository (Repo):** A folder tracked by Git. Everything inside — code, history, branches — is managed by Git. A repo can be local (your machine) or remote (GitHub).

■ Repository Commands

`git init`	**Initialize a new Git repo** in your current folder. Creates a hidden .git folder.
`git init my-project`	**Create a new folder** called my-project and initialize Git inside it.
`git clone <url>`	**Download a remote repo** to your machine. Example: git clone https://github.com/user/repo.git
`git clone <url> folder`	**Clone into a custom folder name** instead of default repo name.
`git status`	**Most used command.** Shows changed, staged, and untracked files. Run this constantly.
`git log`	**Full commit history** with author, date, and message for every commit.
`git log --oneline`	**Compact history.** One line per commit — great for quick overview.
`git log --oneline --graph`	**Visual branch graph** in terminal. Helps understand branching and merges.

■ git status is your best friend — run it before and after every Git operation.

Section 3 — Staging & Committing Changes

■ **How commits work:** Git has 3 areas — **Working Directory** (your files), **Staging Area** (files marked to commit), **Repository** (saved snapshots). Always: **edit** → **git add** → **git commit**.

■ Staging (git add)

`git add .`	**Stage ALL changed files** in current directory. Most commonly used.
`git add <filename>`	**Stage a specific file only.** Example: git add index.html
`git add *.js`	**Stage all .js files** — use wildcards to stage by extension.
`git add -p`	**Interactive staging.** Git shows each change and asks if you want to stage it.
`git restore --staged <file>`	**Unstage a file** without losing your changes.
`git diff`	**See unstaged changes** — line by line what changed but not yet staged.
`git diff --staged`	**See staged changes** — what will go into your next commit.

■ Committing (git commit)

`git commit -m "message"`	**Save a snapshot** with a clear message. Be specific: "Fix login bug on mobile"
`git commit -am "message"`	**Stage + commit in one step** — only for already-tracked files, not new ones.

`git commit --amend -m "new msg"`	**Edit your last commit message** or add forgotten files. Only before pushing.
`git commit --amend --no-edit`	**Add changes to last commit** without changing its message.

■ Write clear messages: "Add user login feature" not "updated stuff". Your future self will thank you.

Section 4 — Branching

■ **What is a Branch?** An independent line of development. Default is **main**. Create a branch to build a feature without touching main code. Merge it back when done. This is how every real company works.

■ Branch Commands

`git branch`	**List all branches.** Current branch has a * star next to it.
`git branch feature-login`	**Create a new branch** called feature-login. Does NOT switch to it.
`git checkout feature-login`	**Switch to an existing branch.** Files now reflect that branch.
`git checkout -b feature-login`	**Create AND switch** to new branch in one command. Most used.
`git switch feature-login`	**Modern way to switch branch** (Git 2.23+). Cleaner alternative to checkout.
`git switch -c feature-login`	**Modern: create and switch** to a new branch.
`git branch -d feature-login`	**Delete merged branch.** Use -D to force-delete unmerged ones.
`git branch -m old-name new-name`	**Rename a branch.**
`git merge feature-login`	**Merge branch into current branch.** First switch to main, then run this.
`git merge --no-ff feature-login`	**Merge with a merge commit** even if fast-forward is possible. Keeps clean history.

■ Always create a new branch for each feature or bug fix. Never code directly on main in a team.

■■ **Merge Conflict:** Happens when two branches changed the same line differently. Git marks conflicts with **<<<<< HEAD** markers in the file. Manually edit the file, choose which version to keep, then git add + git commit to resolve.

Section 5 — Working with Remotes (GitHub)

■ **Remote Repository:** Your project hosted on GitHub. Default remote is called **origin**. Push your local commits to it and pull teammates' changes from it. This is how teams collaborate.

■ Remote Commands

`git remote -v`	**View all remotes** and their URLs (fetch + push).
`git remote add origin <url>`	**Link local repo to GitHub.** Run once after git init for a new project.
`git remote remove origin`	**Remove a remote connection.**
`git remote set-url origin <url>`	**Change the remote URL.** Useful after renaming the repo on GitHub.
`git push origin main`	**Upload commits to GitHub.** This is how your code goes online.

`git push -u origin main`	**Push and set upstream.** After this, just type 'git push' with no arguments.
`git push --force`	**Force push — overwrites remote history.** ■■ Dangerous in shared repos.
`git pull origin main`	**Download + merge latest changes from GitHub.** Do this before starting new work.
`git fetch origin`	**Download changes WITHOUT merging.** Safe way to preview what others pushed.
`git fetch --all`	**Fetch from all remotes.** Useful when working with forks.

■ Rule: Always git pull before you git push. This avoids conflicts with your team's changes.

Section 6 — Undoing Changes

■■ **Made a mistake?** Git lets you undo almost anything. Key: understand WHERE the change is — in working directory, staging area, or already committed.

■■ Undo & Fix Commands

`git restore <file>`	**Discard working directory changes.** Resets file to last commit. Cannot be undone.
`git restore --staged <file>`	**Unstage a file.** Moves it out of staging but keeps your changes.
`git reset --soft HEAD~1`	**Undo last commit, keep changes staged.** Commit disappears but files stay ready.
`git reset --mixed HEAD~1`	**Undo last commit, keep changes unstaged** (default). Files changed, not staged.
`git reset --hard HEAD~1`	**Undo last commit + delete all changes.** ■■ Permanent!
`git revert <commit-id>`	**Safely undo a past commit** by creating a new reverse commit. Safe for shared repos.
`git clean -fd`	**Remove untracked files and folders.** ■■ Permanent.

■ Prefer git revert over git reset --hard in shared repos. Reset rewrites history; revert is safe.

Section 7 — Advanced Git

■ Advanced Commands

`git stash`	**Temporarily save uncommitted work.** Use when switching branches mid-task.
`git stash pop`	**Restore the last stash** and remove it from stash list.
`git stash list`	**See all saved stashes.** You can have multiple stashes.
`git stash apply stash@{0}`	**Apply a specific stash** without removing it from the list.
`git rebase main`	**Reapply your branch commits on top of main.** Cleaner history than merge.

`git rebase -i HEAD~3`	**Interactive rebase — edit last 3 commits.** Squash, rename, reorder commits.
`git cherry-pick <id>`	**Apply a specific commit** from another branch to your current branch.
`git bisect start`	**Binary search to find which commit introduced a bug.**
`git tag v1.0.0`	**Mark a commit as a release version.** Used in production deployments.
`git shortlog -s -n`	**Contribution summary** — who committed how many times.

■ git stash is very useful daily. git rebase -i is what senior devs use to clean history before a PR.

Pro Tips — Work Like a Pro

■ Always Do This	■ Avoid This
Commit often with small, focused changes	One giant commit with everything mixed in
Write clear messages: "Fix null pointer in login"	Vague messages: "fix", "update", "wip"
Always git pull before starting new work	Pushing without pulling first (causes conflicts)
Create a new branch for each feature or bug fix	Coding directly on main branch
Add node_modules, .env, secrets to .gitignore	Committing API keys or passwords to Git
Use git diff before staging to verify changes	Blind git add . without checking what changed
Write descriptive branch names: feature/user-login	Generic names like: test, fix, branch1

Real-World Git Flow (Used in Every Company)

Branch	Purpose	Who touches it
main	Production-ready, stable code	Only via Pull Request
develop	Integration branch for all features	Team merges here first
feature/x	One branch per new feature	Individual developer
hotfix/x	Emergency production bug fix	Senior dev, merged to main
release/v2	Final testing before release	QA / release manager

Daily Developer Workflow — Step by Step

#	Command	What happens
1	`git pull origin main`	Get latest changes from GitHub before starting work
2	`git checkout -b feature/my-task`	Create and switch to a new branch for your task
3	Write your code	Edit files, build features, fix bugs
4	`git status`	Check which files changed
5	`git add .`	Stage all your changes

6	`git commit -m "feat: add login"`	Save a snapshot with a meaningful message
7	`git push origin feature/my-task`	Upload your branch to GitHub
8	Open Pull Request on GitHub	Request teammates to review and merge your code into main

■ **Interview Tip:** When asked about version control — explain the full flow: init → branch → add → commit → push → Pull Request. Mention how to resolve merge conflicts. That alone impresses most interviewers at campus placements.